



浙江大學
ZHEJIANG UNIVERSITY

智能物联网系统设计实验报告

作业三 华为云与 ESP32 的联动

姓名 _____

学号 _____

院系 _____ 信息与工程学院

2026 年 5 月 6 日

目录

1	实验目的与要求	1
2	Arduino 环境配置	1
3	系统设计与代码分析	1
3.1	MQTT 连接与加密	2
3.2	基于 millis() 函数的定时上传任务	2
3.3	传感器测量数据封装与上报	3
3.4	下发命令处理与硬件控制	3
4	实验步骤与成果展示	4
4.1	更改物模型数据结构及设备连接	4
4.2	传感器数据周期上报	5
4.3	云平台下发命令控制 RGB 三色灯	6
5	实验总结与心得体会	7

1 实验目的与要求

本实验为综合性物联网应用开发实验，核心要求如下：

1. **设备连接：**配置 Wi-Fi 与 MQTT 参数，将 ESP32 开发板连接至华为云平台。
2. **数据上报：**采集温度数据（BMP280）和光照数据（光敏电阻），按照 5 秒周期上报至华为云平台。
3. **命令下发：**使用华为云平台命令下发功能，控制 ESP32 端 RGB 三色灯的颜色与开关状态，实现不同的颜色效果。

2 Arduino 环境配置

为实现 ESP32 开发板与各类外设的驱动控制及物联网云端通信，本实验中引入了多个第三方依赖库。具体的库名称、版本要求及功能说明如表 1 所示：

表 1: 软件环境配置表

库名称	版本选择	功能简述
PubSubClient	v2.7.0	MQTT 客户端库。用于连接云平台、向云平台发布传感器数据及接收控制指令。
ArduinoJson	v7.2.0	解析 ESP32 与服务器通信时的 JSON 文件。
Adafruit BMP280	v2.6.8	读取 BMP280 温压传感器测得的温度气压数据，并计算海拔。
Adafruit SSD1306	v2.5.16	控制 OLED 显示屏，提供屏幕初始化、文本显示及图形绘制功能。
Adafruit BusIO	v1.16.1	硬件接口通信库，为上述传感器提供 I2C/SPI 通信支持。
Crypto	v0.4.0	提供加密算法，用于物联网设备接入华为云平台时的加密通信。

3 系统设计与代码分析

本节将对实验提供的 ESP32 示例程序源代码进行简单分析

3.1 MQTT 连接与加密

程序使用了 PubSubClient 库处理 MQTT 通信协议底层。设备接入华为云需满足特定的三元组，即 CLIENT_ID、MQTT_USER 和 MQTT_PASSWORD。在 setupMQTT() 及 reconnectMQTT() 函数中，实现了与华为云的连接逻辑：

```
1 if (client.connect(CLIENT_ID, MQTT_USER, MQTT_PASSWORD)) {  
2     Serial.println("MQTT 连接成功");  
3     // 重新订阅主题  
4     client.subscribe(MQTT_TOPIC_COMMANDS);  
5     client.subscribe(MQTT_TOPIC_GET);  
6 }
```

Code Listing 1: MQTT 客户端连接与主题订阅

其中, MQTT_TOPIC_COMMANDS 对应华为云的 \$oc/devices/{device_id}/sys/commands/# 主题，利用通配符 # 确保能够接收所有由 request_id 的下发命令。

3.2 基于 millis() 函数的定时上传任务

实验要求每 5 秒上报一次数据。如果使用 delay(5000)，会导致单片机在 5 秒内完全阻塞，无法响应云端下发的命令，同时会导致 MQTT 心跳包无法发送而断线。因此，代码采用了非阻塞定时器写法：

```
1 void loop() {  
2     if (!client.connected()) { reconnectMQTT(); }  
3     client.loop(); // 维持 MQTT 心跳  
4  
5     long currentTime = millis();  
6     if (currentTime - lastReportTime > 5000) {  
7         lastReportTime = currentTime;  
8         publishSensorData(); // 执行数据采集与上报  
9     }  
10 }
```

Code Listing 2: 非阻塞定时上报逻辑

该设计在保证 5 秒上报周期的同时，为 client.loop() 留出了执行时间，保障了系统通信稳定性。

3.3 传感器测量数据封装与上报

在 publishSensorData() 函数中，系统首先通过 analogRead(32) 读取光敏电阻值，通过 bmp280.readTemperature() 读取温度值，并实时推送到 OLED 屏幕 (诸多 display 函数)。接着，将数据按照华为云的物模型 JSON 格式进行构建：

```

1 char properties[256];
2 char jsonBuf[512];
3 // 将传感器数值及灯光状态格式化为 JSON 属性字符串
4 sprintf(properties,
5     "\"Temperature\":%.2f,\"Photores\":%d,\"RED\":%d,\"GREEN\":%d
6     ,\"BLUE\":%d,\"Switch\":%d}}}",
7     temperature, photoResistorValue, redValue, greenValue,
8     blueValue, deviceSwitch);
9
10 // 使用 JSON 语言封装传感器数据
11 sprintf(jsonBuf, MQTT_BODY_FORMAT, properties);
12
13 // 发布消息
14 client.publish(MQTT_TOPIC_REPORT, jsonBuf);

```

Code Listing 3: JSON 数据封装与上报

3.4 下发命令处理与硬件控制

当云端下发控制指令时，会触发 callback() 函数。代码使用 ArduinoJson 库对 payload 进行解码，提取 paras 对象中的 RED、GREEN、BLUE 和 Switch 字段：

```

1 if (strstr(topic, MQTT_TOPIC_COMMANDS)) {
2     DynamicJsonDocument doc(256);
3     DeserializationError error = deserializeJson(doc, payload);
4
5     if (error) {
6         Serial.println("JSON 解析失败");
7         return;
8     }
9
10    // 提取 RGB 颜色值和开关状态
11    redValue = doc["paras"]["RED"];
12    greenValue = doc["paras"]["GREEN"];

```

```
13     blueValue = doc["paras"]["BLUE"];
14     deviceSwitch = doc["paras"]["Switch"];
15
16     // 打印接收到的参数
17     Serial.println("解析到的参数:");
18     Serial.printf("RED: %d, GREEN: %d, BLUE: %d, Switch: %d\n",
19                   redValue, greenValue, blueValue, deviceSwitch);
20
21     // 控制LED
22     if (deviceSwitch) {
23         analogWrite(RED_LED, 255 - redValue);
24         analogWrite(GREEN_LED, 255 - greenValue);
25         analogWrite(BLUE_LED, 255 - blueValue);
26     } else {
27         // 关闭LED
28         analogWrite(RED_LED, 255);
29         analogWrite(GREEN_LED, 255);
30         analogWrite(BLUE_LED, 255);
31     }
```

Code Listing 4: 命令处理与硬件控制

命令执行完毕后，系统通过字符串截取从 Topic 中提取出此次命令的 `request_id`，拼接到 `MQTT_TOPIC_CMD_RESPONSE` 主题并回传云端，以告知华为云指令执行成功，以便云平台能够监测到命令下发成功。

4 实验步骤与成果展示

4.1 更改物模型数据结构及设备连接

首先登录华为云 IoTDA 控制台，找到第二次实验中创建的项目产品，导入所提供的第三次课华为云物模型，新增布尔类型的 Switch 属性，用于控制三色灯的开关。随后进入设备详情界面，找到 MQTT 三元组以及设备 ID 与密钥，填入示例代码中。随后打开手机热点并将其名称与密码也填入示例代码中。编译烧录至 ESP32 后，打开串口监视器，可见设备成功连接 Wi-Fi 与 MQTT 服务器。云端显示设备状态为“在线”（如图 1 所示）。



图 1: 设备成功接入华为云 IoTDA 平台（状态显示在线）

4.2 传感器数据周期上报

系统运行后，可观察到单片机上的 OLED 屏幕每 5 秒刷新一次传感器数据。同时，在华为云设备的“消息跟踪”页面，可以看到包含 Temperature 和 Photores 字段的 JSON 报文正以 5 秒为周期上传至云端。

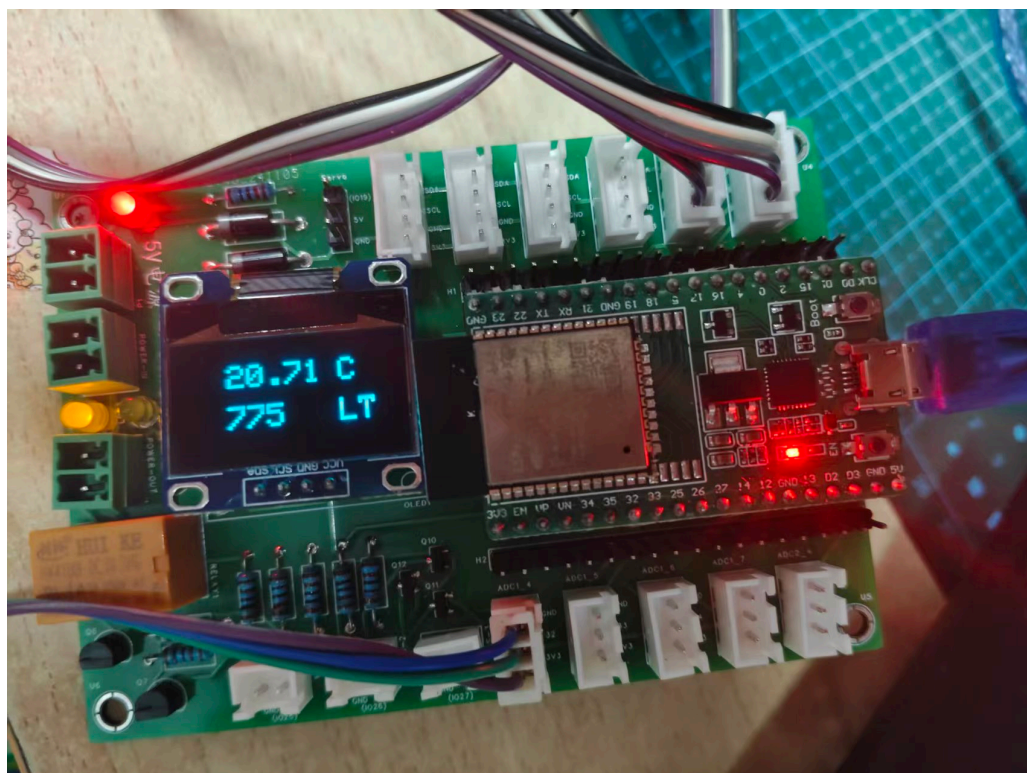


图 2: OLED 屏显示数据

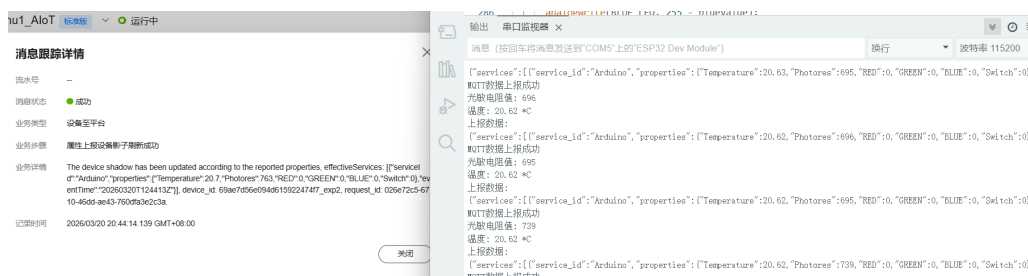


图 3: 云平台成功接收并解析 5 秒周期的上报数据

4.3 云平台下发命令控制 RGB 三色灯

进入华为云平台“命令”页面，选择在物模型中定义的控制逻辑。设置 Switch = 1，并下发特定颜色参数（例如 R=255, G=0, B=0）。观察发现：1. 云平台界面提示“命令下发成功”并收到设备的响应反馈；2. ESP32 本地串口打印出解析成功的 JSON 参数；3. 实验板上的 RGB 灯亮起对应的红色。（云端下发与硬件效果见图 6）



图 4: 云端指令下发界面

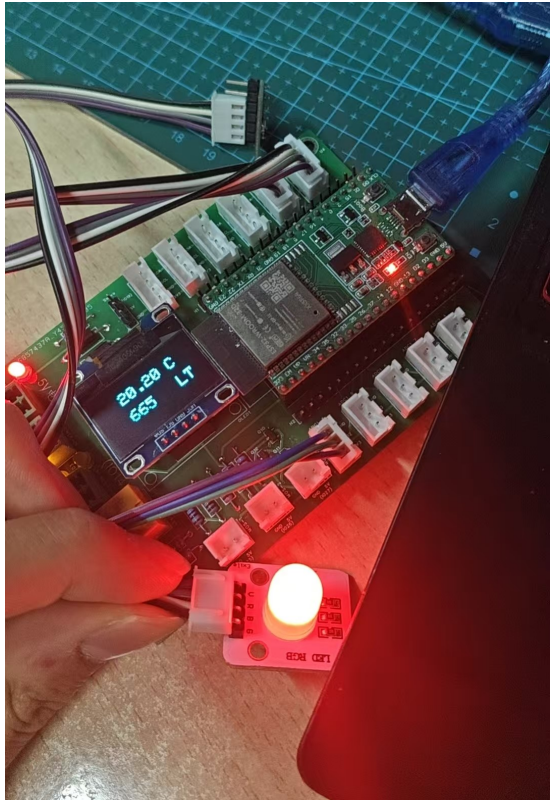


图 5: 开发板 RGB 灯光响应状态

图 6: 华为云端下发控制指令与硬件响应效果

5 实验总结与心得体会

通过本次综合性物联网实验，我实现了从感知层（传感器）、网络层（Wi-Fi 与 MQTT）、到应用层（华为云平台）的物联网开发。

我基本掌握了使用 `ArduinoJson` 库构建与解析物模型报文的方法，以及产品建模、设备注册等基础功能，更体会到了云端控制设备的强大之处——开发者无需在实体设备中编写繁琐的逻辑判断代码，可利用云平台实现设备联动与数据通信，提升了系统的灵活性与可维护性。本次实验为我的后续开发更加复杂的智能物联网项目打下了坚实的技术基础。